

Prompt:

8/17/2026, 8:00:03 PM

<https://github.com/tabassam823/TradeProject.git>

bisakah kau melihat isi dari repository ini? bisakah kau menjelaskan padaku algoritma utama yang bekerja pada kode yang ada pada repo tersebut?

Response:

8/17/2026, 8:00:17 PM

Searching the web

Searching github.com

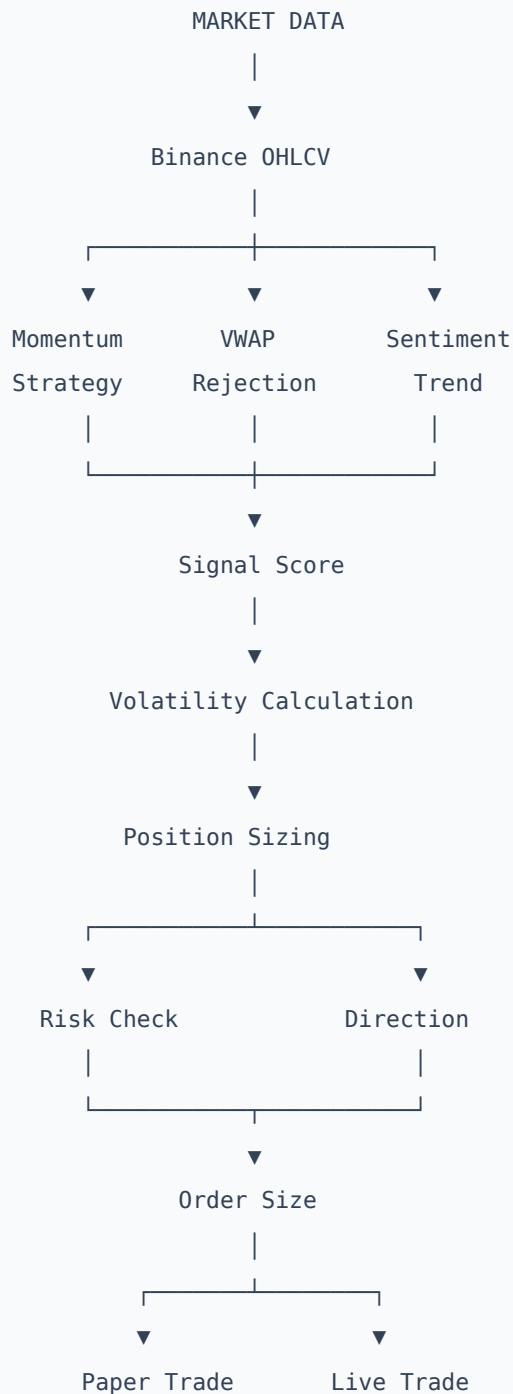
Worked for 10s

Ya. Aku bisa melihat repository tersebut, dan aku sudah menelusuri struktur utama serta kode strategi di dalamnya. Repository ini adalah `tabassam823/TradeProject`, dan saat ini branch `main` berisi engine backtesting, paper trading, live trading, tiga strategi, position sizing berbasis volatilitas, serta integrasi Binance dan OpenBB. ([GitHub](#))

Yang menarik: **kode aktualnya tidak persis sama dengan narasi "AI self-improving trading agent" yang ada di `workflow.md`**. Implementasi trading yang benar-benar berjalan saat ini lebih tepat disebut **multi-strategy quantitative trading system dengan volatility-targeted risk management**. ([GitHub](#))

1. Gambaran besar algoritamanya

Secara konseptual, alurnya adalah:



Ada tiga lapisan yang sebaiknya kamu pisahkan secara mental:

1. **Signal generation** → "Apakah saya long/short?"
2. **Risk & position sizing** → "Seberapa besar posisi saya?"
3. **Execution** → "Bagaimana order dikirim ke Binance?"

Ini penting karena **algoritma trading-nya bukan sekadar satu indikator**.

2. Tiga strategi utama

`strategy_config.json` mengaktifkan tiga strategi:

Strategi	Ide utama	Signal
Multi-Horizon Momentum	Trend-following	-4 sampai +4
VWAP Rejection	Mean reversion	-2, 0, +2
Sentiment-Filtered Trend	EMA + news sentiment	-3, 0, +3

Konfigurasi saat ini menggunakan horizon momentum `[5, 10, 21, 42]`, VWAP window 24 candle dengan $\pm 2\sigma$, serta EMA 10/30 untuk sentiment trend. ([GitHub](#))

3. Strategi #1 - Multi-Horizon Momentum

Ini menurutku merupakan **strategi paling sederhana sekaligus paling penting** di repository.

Kode:

`src/strategies/momentum.py`

Algoritmanya melihat empat horizon:

$$\tau \in \{5, 10, 21, 42\}$$

Kemudian untuk masing-masing horizon dihitung:

$$\Delta P_{\tau} = P_t - P_{t-\tau}$$

Lalu:

$$s_{\tau} = \begin{cases} +1, & P_t > P_{t-\tau} \\ -1, & P_t < P_{t-\tau} \\ 0, & P_t = P_{t-\tau} \end{cases}$$

Kemudian semua signal dijumlahkan:

$$S_{\text{momentum}} = \sum_{\tau \in \{5, 10, 21, 42\}} s_{\tau}$$

Sehingga:

$$S_{\text{momentum}} \in [-4, +4]$$

Implementasinya memang persis seperti itu: setiap horizon memberikan `+1`, `-1`, atau `0`, kemudian dijumlahkan. ([GitHub](#))

Contoh

Misalnya:

```
5 candle → harga naik → +1
10 candle → harga naik → +1
21 candle → harga turun → -1
42 candle → harga naik → +1
```

Maka:

$$S = 1 + 1 - 1 + 1 = 2$$

Artinya bias sistem adalah **long**.

Jadi ini bukan machine learning.

Ini adalah **multi-time-horizon trend consensus**.

4. Strategi #2 - VWAP Mean Reversion

Kode:

```
src/strategies/vwap_rejection.py
```

Ini jauh lebih menarik secara matematis.

Pertama dihitung typical price:

$$TP_t = \frac{H_t + L_t + C_t}{3}$$

Kemudian VWAP:

$$VWAP = \frac{\sum_t TP_t V_t}{\sum_t V_t}$$

Kemudian deviasi standar berbobot volume:

$$\sigma = \sqrt{\frac{\sum_t (TP_t - VWAP)^2 V_t}{\sum_t V_t}}$$

Kemudian dibuat dua batas:

$$U = VWAP + 2\sigma$$

$$L = VWAP - 2\sigma$$

Karena konfigurasi menggunakan:

```
vwap_window = 24
num_std = 2
```

maka sistem melihat area $\pm 2\sigma$ di sekitar VWAP. ([GitHub](#))

Entry long

Jika:

$$P_{t-1} \leq L$$

tetapi kemudian:

$$P_t > L$$

maka:

$$S = +2$$

Interpretasinya:

harga sebelumnya berada di bawah lower value area, lalu kembali masuk → dianggap rejection → long.

Sebaliknya:

$$P_{t-1} \geq U$$

dan:

$$P_t < U$$

memberikan:

$$S = -2$$

yaitu short.

Jadi secara konsep:



Ini **mean-reversion strategy**, bukan trend following.

5. Strategi #3 - Sentiment-Filtered Trend

Kode:

```
src/strategies/sentiment_trend.py
```

Di sini digunakan dua EMA:

$$EMA_{fast} = EMA_{10}$$

dan

$$EMA_{slow} = EMA_{30}$$

Kemudian sistem mengambil sentiment berita melalui OpenBB.

Untuk long:

$$EMA_{10} > EMA_{30}$$

dan:

$$Sentiment \geq -0.2$$

maka:

$$S = +3$$

Untuk short:

$$EMA_{10} < EMA_{30}$$

dan:

$$Sentiment \leq +0.2$$

maka:

$$S = -3$$

Jika tidak memenuhi kondisi:

$$S = 0$$

([GitHub](#))

Jadi ide dasarnya:

Technical trend + fundamental/news confirmation.

6. Bagian yang menurutku paling penting: Position Sizing

Justru menurutku **bagian paling "quantitative"** dari repository ini bukan **signal generation**-nya, tetapi **risk engine**-nya.

Kode:

```
src/core/position_sizer.py
```

Volatilitas dihitung dari return:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

kemudian:

$$\sigma_t = Std(r_{t-20}, \dots, r_t)$$

dengan lookback 21 candle. ([GitHub](#))

Kemudian volatilitas digunakan untuk menentukan ukuran posisi.

Ini berarti:

signal menentukan arah, volatilitas menentukan seberapa agresif posisi.

7. Risk budget

Ada mekanisme tiered risk:

Jika modal:

$$C < 100$$

maka:

$$R = \$1$$

Jika:

$$C \geq 100$$

maka:

$$R = 0.01C$$

Jadi dengan modal \$10,000:

$$R = 0.01(10000) = \$100$$

Artinya sistem mencoba membatasi kerugian per trade sekitar \$100. ([GitHub](#))

Ini jauh lebih masuk akal daripada:

"Saya punya signal kuat → masuk 100% modal."

8. Stop loss dan take profit juga dihitung matematis

Ini bagian yang sangat menarik.

Repository menggunakan konsep expected value:

$$E[X] = Wp - (1 - W)$$

secara konseptual, dengan payoff ratio minimum:

$$b_{\min} = \frac{1 - W}{W}$$

Kode kemudian mengambil:

$$b_{\text{target}} = \max(1.2b_{\min}, 1.5)$$

Kemudian:

$$SL\% = \max(1.5\sigma, 1\%)$$

dan:

$$TP\% = SL\% \times b_{\text{target}}$$

([GitHub](#))

Jadi sistem tidak sekadar mengatakan:

"Stop loss = 2%."

Tetapi secara prinsip:

$$\text{Volatility} \rightarrow SL \rightarrow TP$$

9. Position size kemudian diturunkan dari risk

Misalkan:

- harga entry = P
- stop loss = P_{SL}

- risiko yang diizinkan = R

Jarak stop:

$$D = |P - P_{SL}|$$

Jumlah unit:

$$N = \frac{R}{D}$$

Itulah yang dilakukan kode:

```
units = risk_dollar / sl_distance
```

([GitHub](#))

Contoh sederhana:

$$P = 100$$

$$P_{SL} = 98$$

sehingga:

$$D = 2$$

Jika risiko:

$$R = \$100$$

maka:

$$N = \frac{100}{2} = 50$$

Jika stop loss tersentuh:

$$50 \times 2 = \$100$$

Jadi **position sizing berasal dari risk budget, bukan dari confidence signal semata.**

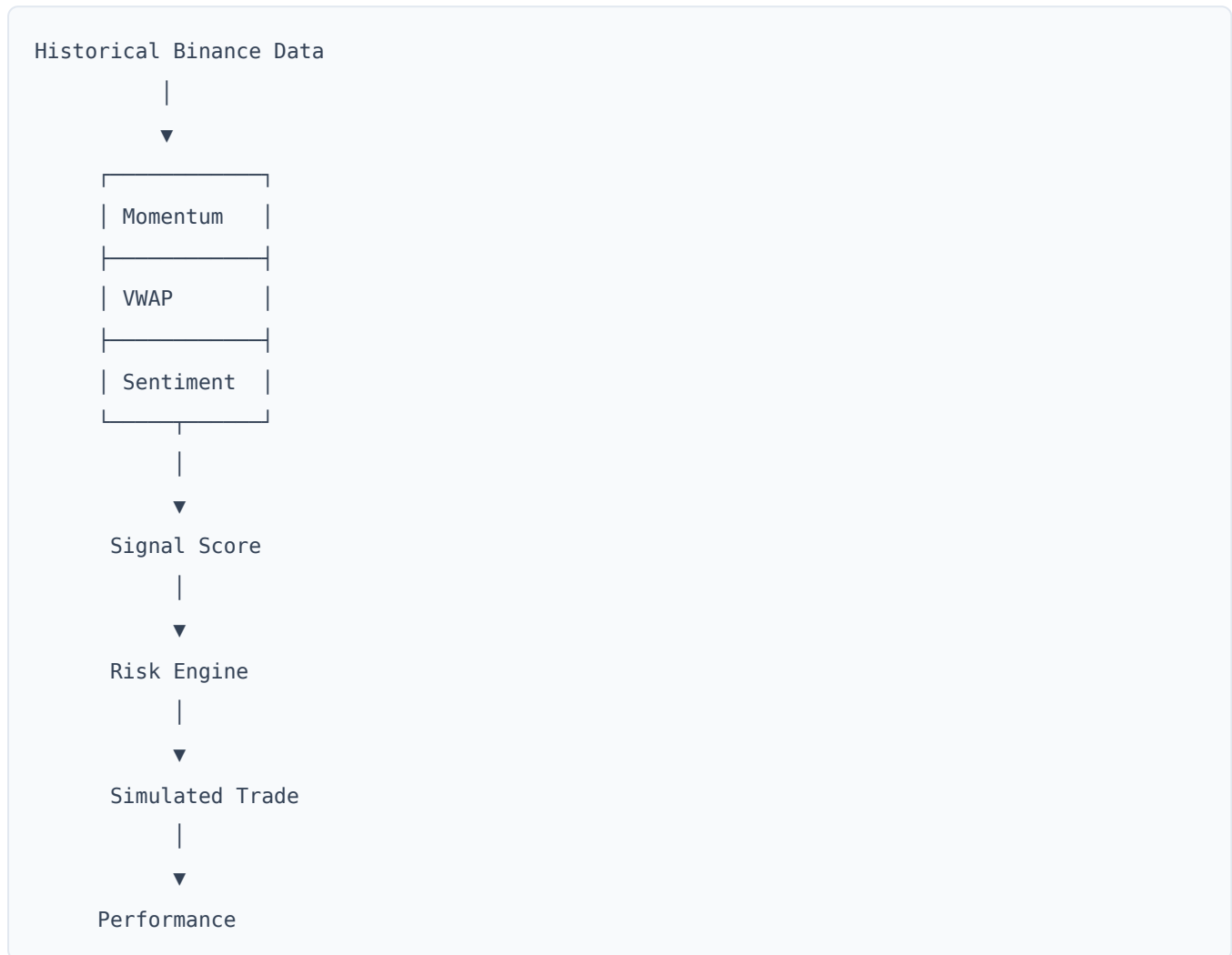
10. Backtesting

Backtester utamanya ada di:

```
src/backtester.py
```

Ia mengambil historical OHLCV dari Binance, kemudian menjalankan ketiga strategi tersebut. ([GitHub](#))

Strukturnya:



Yang perlu diperhatikan:

backtester mengevaluasi strategi secara terpisah, bukan sebuah neural network yang belajar dari hasil backtest.

11. Paper trading

Paper trader menggunakan data Binance secara real-time dan mensimulasikan eksekusi. Kode bahkan menjelaskan bahwa engine ini mengambil ticker real-time setiap jam dan menyimpan virtual ledger ke

```
logs/paper_ledger.json
```

 ([GitHub](#))

Jadi:

Market Data → Strategy → Risk → Virtual Order

Tidak ada uang sungguhan.

12. Live trading

Live trader menghubungkan:

```
Binance API
+
OpenBB
+
Position Sizer
+
Momentum Strategy
```

dan kemudian dapat mengirim order ke Binance. ([GitHub](#))

Menariknya, ada **circuit breaker** berbasis OpenBB sentiment:

```
Market
|
▼
Signal
|
▼
Sentiment/Circuit Breaker
|
|— BLOCK → jangan trade
|
|— PASS
|
▼
Position Sizing
|
▼
Binance Order
```

Tetapi ada satu hal yang sangat penting:

LiveTrader saat ini hanya memilih `MultiHorizonMomentumStrategy`

Di constructor-nya terlihat:

```
self.strategy = MultiHorizonMomentumStrategy(...)
```

Jadi walaupun konfigurasi memiliki tiga strategi, **live execution tidak menjalankan ketiganya sebagai ensemble**. ([GitHub](#))

Ini perbedaan penting antara:

"repository memiliki tiga strategi"

dan:

"live trading menjalankan tiga strategi."

Yang kedua **belum sepenuhnya benar** berdasarkan kode saat ini.

13. Lalu apakah ini "AI Trading Agent"?

Menurutku: belum dalam arti machine-learning/AI agent yang kuat.

Ini lebih tepat disebut:

Quantitative Trading Engine

dengan komponen:

Rule-Based Strategies + Risk Management + Execution

Memang `workflow.md` berbicara tentang:

- self-improving agent
- feedback loop
- autonomous optimization
- perubahan parameter
- scientific method
- Sharpe ratio
- AI agent

([GitHub](#))

Tetapi **kode trading** yang sekarang tidak menunjukkan adanya optimizer/ML loop yang benar-benar **mengubah parameter strategi** berdasarkan hasil trading.

Dengan kata lain:

DOKUMENTASI

AI Agent

↓

Self Improvement

↓

Change Parameters

↓

Evaluate

↓

New Baseline

sedangkan implementasi aktual lebih dekat ke:

Fixed Parameters

↓

Rule-Based Strategy

↓

Signal

↓

Risk Management

↓

Trade

Itu perbedaan yang cukup besar.

14. Jadi algoritma utamanya sebenarnya apa?

Kalau aku harus merangkum repository ini dalam satu persamaan konseptual:

$$\text{Trade} = f(\text{Momentum, VWAP, Sentiment, Volatility, Risk Budget})$$

Lebih detail:



dengan:

$$\text{Signal Generation} = \begin{cases} \text{Multi-Horizon Momentum} \\ \text{VWAP Mean Reversion} \\ \text{EMA + Sentiment} \end{cases}$$

dan:

$$\text{Risk Engine} = \text{Volatility} + \text{Risk Budget} + \text{SL/TP} + \text{Position Size}$$

15. Ada satu hal yang menurutku sangat relevan dengan proyekmu

Kalau tujuanmu adalah **mengembangkan repository ini lebih lanjut**, terutama mengingat kita sebelumnya membahas algoritma trading otomatis dan QAOA/VQE, maka repository ini sebenarnya menyediakan tempat yang sangat jelas untuk memasukkan algoritma optimasi yang lebih canggih.

Saat ini:

$$\boxed{\text{Strategy} \rightarrow \text{Signal} \rightarrow \text{Position Size}}$$

Tetapi bisa dikembangkan menjadi:

$$\boxed{\text{Strategies} \rightarrow \text{Portfolio/Signal Optimization} \rightarrow \text{Position Allocation} \rightarrow \text{Execution}}$$

Misalnya tiga strategi menghasilkan:

$$S = \begin{bmatrix} S_{\text{momentum}} \\ S_{\text{VWAP}} \\ S_{\text{sentiment}} \end{bmatrix}$$

Kemudian kita mencari bobot:

$$w = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}$$

sehingga:

$$S_{\text{final}} = w_1 S_{\text{momentum}} + w_2 S_{\text{VWAP}} + w_3 S_{\text{sentiment}}$$



Nah, **di sinilah sebenarnya QAOA/VQE atau metode optimasi lainnya bisa masuk secara natural**-bukan menggantikan indikator, tetapi mengoptimalkan **allocation/weighting/portfolio decision** di atas signal yang sudah tersedia.

Dan itu menurutku jauh lebih menarik daripada sekadar mengganti salah satu indikator dengan "AI".

Kalau kamu mau, langkah berikutnya yang paling berguna adalah **aku bedah `backtester.py` baris demi baris dan turunkan seluruh persamaan matematis yang sebenarnya dipakai repository ini**, termasuk bagaimana `signal score → volatility → SL/TP → leverage → position size → PnL` bekerja. Itu akan memberi kita gambaran yang sangat jelas tentang **algoritma trading sebenarnya**, bukan hanya dokumentasinya. ([GitHub](#))

Sources:

- [GitHub - tabassam823/TradeProject · GitHub](#)